



PRÉ-RENTRÉE 2026-2027

Démarrage rapide

Préparer le stage sans friction

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document enseignant

Avant le premier jour

- ☐ imprimer 08_PACKS_PDF/1re_nsi_PACK_ELEVE_COMPLET.pdf en recto simple ;
- ☐ conserver le pack enseignant hors de la vue des élèves ;
- ☐ imprimer séparément les deux dossiers nominatifs ;
- ☐ installer Python 3.11+ et Thonny sur les deux postes ;
- ☐ décompresser le pack de code commun et le pack personnel de chaque élève ;
- ☐ ne jamais copier 1re_nsi_CODE_ENSEIGNANT.zip sur un poste élève ;
- ☐ vérifier l'ouverture de mesures_capteurs.csv ;
- ☐ créer un dossier de sauvegarde pour chaque élève ;
- ☐ ouvrir le tableau de bord enseignant ;
- ☐ reporter les objectifs individuels.

Répartition des parcours

Élève	Parcours	Principe
Ahmad BELDI	Fondations guidées	traçage obligatoire, code à compléter, feedback fréquent et retrait progressif des aides
Ahmed BENHADJ SALEM	Fiabilisation et autonomie	débogage, cas limites, contrats, tests, justification et QCM chronométré

Les fiches communes ne portent aucun nom. Le parcours est indiqué dans le dossier nominatif et par l'enseignant.

Préparer chaque séance

Séance	Fichier Python	Support principal	Production attendue
1	s1_variables_conditions_starter.py	tables de trace et boîtes mémoire	premier programme interactif commenté
2	s2_boucles_starter.py	cartes range , compteur et accumulateur	outils d'analyse d'une liste

Séance	Fichier Python	Support principal	Production attendue
3	s3_fonctions_tests_starter.py	cartes paramètre/ argument/retour	fonctions documentées avec assertions
4	s4_structures_csv_starter.py	cartes listes/ dictionnaires/CSV	chargement et traitement de données capteurs
5	projet_final_starter_ahmad.py ou cahier des charges Ahmed	tris, dichotomie et grille de tests	mini-projet final testé et présenté

Routine des 120 minutes

1. rituel et prédiction ;
2. apport théorique bref ;
3. traçage papier ;
4. programmation ;
5. tests et débogage ;
6. différenciation ;
7. jalon du projet ;
8. exit ticket.

Contrôles de fin de séance

- ☐ le code élève est sauvegardé et versionné ;
- ☐ le corrigé est resté séparé ;
- ☐ l'aide maximale a été consignée ;
- ☐ au moins un test normal et un cas limite ont été exécutés ;
- ☐ l'élève a expliqué une erreur ;
- ☐ le tableau de bord est mis à jour ;
- ☐ le travail à reprendre avant la séance suivante est explicite.



PRÉ-RENTRÉE 2026-2027

Stage d'entrée en Première NSI

Maîtrise opérationnelle de Python

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Groupe : Ahmad BELDI et Ahmed BENHADJ SALEM

1. Positionnement et décision pédagogique

1.1 Finalité réelle du stage

Le stage est **centré sur la maîtrise opérationnelle de Python**, car la NSI n'a pas été enseignée auparavant dans le parcours scolaire ordinaire d'Ahmad BELDI. Il ne s'agit pas de reproduire en dix heures l'intégralité d'une année de Première NSI. La cible réaliste et exigeante est la suivante :

rendre disponibles les constructions Python et les méthodes de programmation nécessaires pour suivre la Première NSI sans blocage, comprendre un programme, en écrire un, le décomposer, le tester, le corriger et expliquer son fonctionnement.

Le programme officiel de Première NSI en vigueur reste celui du BO spécial n° 1 du 22 janvier 2019. Il privilégie Python 3, sans faire de l'expertise dans un langage une fin en soi. Il attend notamment les séquences, affectations, conditions, boucles, fonctions, spécifications, assertions, jeux de tests, types construits, tables de données et algorithmes classiques.

1.2 Limite assumée

Les thèmes Web, réseaux, architecture et systèmes figurent au programme annuel, mais ne constituent pas l'objet principal de ce stage. Ils apparaissent seulement dans les rituels, le mémento et la feuille de route annuelle. La totalité des dix heures est structurée autour de Python et de l'algorithmique.

1.3 Effectif et profils

Élève	Situation	Données disponibles	Décision pédagogique
Ahmad BELDI	Passage de Seconde en Première ; aucune NSI formellement enseignée	13/18 questions traitées ; programmation 22,2 % ; représentation binaire, Web et données en tables à consolider ; réseaux à situer	Parcours Fondations guidées : modèle mental de l'exécution, traçage, syntaxe, indexation, fonctions et tests

Élève	Situation	Données disponibles	Décision pédagogique
Ahmed BENHADJ SALEM	Candidat individuel ; a déjà présenté une première partie sans réussite	Test Première : 18/18 traitées, quatre domaines solides, programmation 55,6 % ; test Terminale : six domaines solides mais erreur d'accumulateur	Parcours Fiabilisation et autonomie : corriger les automatismes fragiles, coder sous contrainte, tester, documenter et expliquer

1.4 Conceptions prioritaires

Ahmad BELDI

- pense qu'une affectation conserve un lien dynamique entre deux variables ;
- interprète `x = x + 1` comme un nom de variable ;
- oublie le cas d'égalité dans la négation d'une condition ;
- interprète `range(3)` comme une seule exécution ;
- commence l'indexation d'une liste à 1 ;
- ne distingue pas la longueur d'une liste du dernier indice ;
- confond paramètre, valeur renvoyée et portée locale ;
- ne connaît pas le retour implicite `None`.

Ahmed BENHADJ SALEM

- maîtrise une grande partie des repères théoriques, mais conserve des erreurs fortement assurées ;
- confond parfois la négation logique ;
- inverse l'usage de `for` et `while` ;
- confond `len(L)` et l'indice du dernier élément ;
- ne stabilise pas encore le retour `None` ;
- a commis une erreur d'accumulation en confondant nombre d'itérations et somme produite ;
- doit transformer une connaissance déclarative en code fiable, testé et explicable.

2. Référentiel Python de Première NSI

2.1 Matrice de couverture

Capacité attendue pendant l'année	Couverture dans le stage	Séance	Niveau visé à la sortie
Séquences, affectations, expressions et types	Complète	1	Autonome

Capacité attendue pendant l'année	Couverture dans le stage	Séance	Niveau visé à la sortie
Booléens, comparaisons, <code>and</code> , <code>or</code> , <code>not</code> , conditions	Complète	1	Autonome
Boucles bornées <code>for</code> , <code>range</code>	Complète	2	Autonome
Boucles non bornées <code>while</code> , terminaison	Complète	2	Autonome sur cas simples
Compteurs, accumulateurs, recherche de minimum/maximum	Complète	2	Autonome
Fonctions, paramètres, arguments, <code>return</code> , <code>None</code> , portée	Complète	3	Autonome
Préconditions, postconditions, assertions, tests	Complète	3	Autonome sur fonctions simples
Documentation et lecture de documentation	Complète	3	Utilisable
Chaînes, tuples, listes, tableaux, compréhensions	Complète	4	Autonome
Dictionnaires, clés/valeurs/items	Complète	4	Autonome
Tableaux de tableaux	Introduite	4	Guidé
Import CSV, sélection, tri, cohérence, doublons	Complète	4-5	Autonome sur un fichier simple
Recherche séquentielle, extremum, moyenne	Complète	2 et 5	Autonome
Tri par insertion et par sélection	Un algorithme écrit ; l'autre analysé	5	Guidé puis transférable
Recherche dichotomique	Introduite et tracée	5	Compréhension et adaptation
Coût linéaire, quadratique, logarithmique	Intuition structurée	5	Comparaison qualitative
Invariant et variant de boucle	Initiation	2 et 5	Savoir les expliquer sur un exemple
Bibliothèques Python	Utilisation standard de <code>csv</code> et de <code>math</code>	3-5	Lire une documentation simple
K plus proches voisins et glouton	Panorama et feuille de route	5	Non évalué au stage

2.2 Compétences transversales

- décomposer un problème en sous-problèmes ;
- écrire un algorithme avant le code lorsque la tâche est complexe ;
- tracer manuellement un programme ;
- expliquer le rôle de chaque variable ;
- spécifier les entrées et sorties ;
- choisir des tests normaux, limites et invalides ;
- corriger un bug à partir d'une observation ;
- lire et réutiliser du code existant ;
- présenter oralement une solution.

3. Objectifs de sortie

À l'issue des cinq séances, l'élève doit pouvoir :

1. lire un programme court et produire une table de trace exacte ;
2. distinguer affectation, comparaison et calcul ;
3. manipuler les types `int`, `float`, `str`, `bool` et `None` ;
4. écrire une condition correcte et sa négation ;
5. choisir entre `for` et `while` ;
6. utiliser correctement `range`, un compteur et un accumulateur ;
7. écrire une fonction avec paramètres et valeur renvoyée ;
8. distinguer `print` et `return` ;
9. écrire une docstring, une précondition et des assertions de test ;
10. manipuler listes, tuples et dictionnaires ;
11. parcourir, filtrer et transformer une table CSV ;
12. écrire une recherche séquentielle et une recherche d'extremum ;
13. comprendre les principes d'un tri et d'une recherche dichotomique ;
14. produire un petit programme découpé en fonctions, documenté et testé ;
15. expliquer ses choix et contrôler son résultat.

4. Progression des cinq séances

Séance	Axe	Théorie	Pratique principale	Livrable
1	Variables, types, booléens et conditions	État mémoire, affectation, expression, type, logique	Classificateur de mesure et table de trace	Fiche « lire et écrire un programme simple »

Séance	Axe	Théorie	Pratique principale	Livrable
2	Boucles et schémas algorithmiques	<code>for</code> , <code>while</code> , compteur, accumulateur, invariant, terminaison	Analyse d'une série de mesures	Bibliothèque de parcours de listes
3	Fonctions, contrats, tests et débogage	Paramètre, argument, retour, portée, <code>None</code> , assertions	Module de fonctions testées	Gabarit de fonction propre et testée
4	Listes, tuples, dictionnaires et CSV	Mutabilité, indexation, compréhensions, enregistrements	Import et filtrage de <code>mesures_capteurs.csv</code>	Analyseur de table version 1
5	Algorithmes et mini-projet	Recherche, tri, dichotomie, coût	Projet final d'analyse de données	Programme documenté + bilan individuel

5. Déroulés synthétiques

Séance 1 - Variables, types, booléens et conditions

- 0-10 min : installation, règles de nommage et test de l'environnement ;
- 10-25 min : table de trace sur les affectations ;
- 25-45 min : types, opérateurs et conversions ;
- 45-65 min : expressions booléennes, conditions et négations ;
- 65-70 min : pause ;
- 70-95 min : programmation guidée ;
- 95-110 min : parcours différenciés ;
- 110-118 min : tests et revue croisée ;
- 118-120 min : exit ticket.

Séance 2 - Boucles et schémas algorithmiques

- 0-10 min : rituel de traçage ;
- 10-30 min : `range`, bornes et indices ;
- 30-50 min : compteurs et accumulateurs ;
- 50-65 min : choix `for/while`, terminaison ;
- 65-70 min : pause ;
- 70-100 min : atelier « analyse d'une série » ;
- 100-112 min : différenciation ;
- 112-120 min : invariant, contrôle et exit ticket.

Séance 3 - Fonctions, contrats et tests

- 0-10 min : rituel ;
- 10-30 min : paramètres, arguments et portée ;
- 30-48 min : `return`, `print` et `None` ;
- 48-65 min : docstrings, préconditions et postconditions ;
- 65-70 min : pause ;
- 70-100 min : écriture d'un module de fonctions ;
- 100-112 min : jeux de tests et débogage ;
- 112-120 min : revue de code et exit ticket.

Séance 4 - Structures de données et tables

- 0-10 min : rituel ;
- 10-28 min : chaînes, tuples, listes et indexation ;
- 28-45 min : mutation, alias et copie ;
- 45-65 min : dictionnaires et enregistrements ;
- 65-70 min : pause ;
- 70-100 min : lecture CSV et filtrage ;
- 100-112 min : tri, doublons et cohérence ;
- 112-120 min : synthèse et exit ticket.

Séance 5 - Algorithmes et projet final

- 0-10 min : quiz cumulatif ;
- 10-27 min : recherche séquentielle et extremum ;
- 27-43 min : tri par insertion ;
- 43-58 min : recherche dichotomique et coûts ;
- 58-63 min : présentation du projet ;
- 63-68 min : pause ;
- 68-105 min : réalisation du projet final ;
- 105-115 min : démonstration orale et tests ;
- 115-120 min : bilan et plan de travail.

6. Différenciation des deux élèves

6.1 Tronc commun

Le tronc commun représente environ 70 % du temps. Les deux élèves travaillent les mêmes notions et le même projet afin de favoriser les explications croisées.

6.2 Ahmad BELDI - parcours Fondations guidées

- tables de trace obligatoires avant exécution ;
- code à trous puis code autonome ;
- fonctions courtes, une difficulté à la fois ;
- aide graduée sur `range`, indices, `len`, paramètres et `return` ;
- tests fournis au départ, puis à compléter ;
- projet final avec squelette détaillé.

Indicateurs

- 4 tables de trace exactes sur 5 ;
- aucune confusion affectation/comparaison ;
- `range` et indexation maîtrisés ;
- fonction avec paramètre et retour correctement écrite ;
- liste parcourue sans erreur de borne ;
- projet exécuté sans erreur et expliqué.

6.3 Ahmed BENHADJ SALEM - parcours Fiabilisation et autonomie

- même tronc commun, mais moins d'étapes fournies ;
- cas limites et entrées invalides ;
- obligation de rédiger les tests avant certaines fonctions ;
- tâches chronométrées courtes ;
- comparaison de deux algorithmes ;
- projet final avec spécification, sans squelette détaillé ;
- oral de justification inspiré des exigences pratiques du baccalauréat.

Indicateurs

- négations logiques correctes ;
- choix `for/while` justifié ;
- accumulateur fiable ;
- distinction `len(L)` / dernier indice ;
- retour `None` compris ;
- tests incluant cas limite ;
- projet final documenté et soutenu sans dépendance forte à l'enseignant.

7. Évaluation

7.1 Avant

- test initial NSI déjà administré ;
- mini-diagnostic pratique de 25 minutes ;
- entretien oral de cinq minutes par élève.

7.2 Pendant

- cinq rituels ;
- cinq exit tickets ;
- journal des aides ;
- validation progressive des fichiers Python ;
- revue croisée des tests.

7.3 Finale

- quiz théorique court ;
- tâche pratique sur ordinateur ;
- mini-projet ;
- présentation orale de deux minutes ;
- grille de compétences, sans note de classement.

8. Feuille de route après le stage

Le stage couvre le socle Python. La poursuite de Première devra réactiver puis approfondir :

- binaire, hexadécimal, complément à deux, flottants et encodage ;
- listes de listes, dictionnaires et tables ;
- tris, dichotomie, k plus proches voisins et algorithmes gloutons ;
- Web, HTTP, formulaires et client-serveur ;
- architecture de von Neumann, réseau et système d'exploitation ;
- projets en binôme avec documentation et tests.

9. Références officielles

- Ministère de l'Éducation nationale, **Programme de numérique et sciences informatiques de première générale**, BO spécial n° 1 du 22 janvier 2019, NOR MENE1901633A.
- Éduscol, **Programmes et ressources en numérique et sciences informatiques - voie générale**, mise à jour février 2026.
- Les ressources Éduscol sur les types construits, les tables de données, la mise au point de programmes testés et la recherche dichotomique ont servi de contrôle de cohérence.



PRÉ-RENTRÉE 2026-2027

Guide du formateur

Stage Première NSI - maîtrise de Python

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document enseignant confidentiel

1. Principes de conduite

1.1 Posture

- ne jamais supposer que la réponse à un QCM prouve une maîtrise pratique ;
- faire tracer avant d'exécuter ;
- demander « quelle valeur possède chaque variable à cet instant ? » ;
- faire écrire des tests avant de multiplier les exercices ;
- distinguer erreur de syntaxe, erreur d'exécution et erreur logique ;
- ne pas corriger immédiatement une certitude erronée : provoquer d'abord un contre-exemple ;
- valoriser un code simple, lisible et testé plutôt qu'un code court mais opaque.

1.2 Organisation matérielle

- un ordinateur par élève ;
- Python 3 installé, idéalement via Thonny ou un environnement équivalent ;
- aucun paquet externe requis ;
- dossier 06_CODE copié localement ;
- mode hors ligne possible ;
- vidéoprojecteur pour les revues de code ;
- feuilles de trace et cartes d'aide imprimées.

1.3 Rôles en binôme

Pour les activités communes, alterner toutes les 12 à 15 minutes :

- **pilote** : écrit le code ;
- **navigateur** : lit la spécification, anticipe le résultat, contrôle les tests ;
- puis inversion des rôles.

2. Diagnostic de départ

Ahmad BELDI

Le bilan montre une programmation encore fragile : 13 questions traitées sur 18, programmation à 22,2 %, avec des erreurs sur l'affectation, l'incrémentation, les négations, `range`, l'indexation, `len`, les paramètres et `None`. La priorité n'est pas la vitesse mais la construction d'un modèle mental correct.

Ahmed BENHADJ SALEM

Le bilan de Première montre quatre domaines solides mais une programmation à 55,6 %. Les erreurs portent sur la négation, le choix `for/while`, `len` et `None`. Un bilan de niveau Terminale met aussi en évidence une erreur d'accumulateur. L'enjeu est la fiabilité sous contrainte, pas la découverte théorique.

3. Routine de séance

1. rituel sans ordinateur ;
2. prédiction du comportement d'un code ;
3. exécution et comparaison ;
4. apport théorique court ;
5. programmation guidée ;
6. parcours différencié ;
7. tests et revue ;
8. exit ticket et mise à jour du tableau de bord.

4. Barème de compétences

Niveau	Description
0	non situé
1	ne sait pas commencer ou applique une conception erronée
2	réussit avec aide ou sur un cas familier
3	réussit seul, explique et teste

Niveau	Description
4	adapte à un problème nouveau et aide à valider une solution

5. Déroulé des séances

Séance 1

- **objectif commun** : comprendre l'affectation et écrire un programme conditionnel ;
- **Ahmad** : trace guidée, une variable par ligne, code à trous ;
- **Ahmed** : conditions composées, négations et cas limites ;
- **preuve attendue** : deux tables de trace exactes et un programme avec trois branches testé sur les bornes.

Séance 2

- **objectif commun** : maîtriser les boucles et les schémas compteur/accumulateur ;
- **Ahmad** : `range`, indices, compteur, somme ;
- **Ahmed** : `while`, terminaison, accumulateur, tests sur liste vide ;
- **preuve attendue** : programme calculant somme, moyenne, maximum et nombre de valeurs répondant à un critère.

Séance 3

- **objectif commun** : écrire des fonctions propres et testées ;
- **Ahmad** : paramètre, argument, `return`, `None` ;
- **Ahmed** : préconditions, postconditions, assertions et couverture de tests ;
- **preuve attendue** : module de quatre fonctions et au moins huit assertions.

Séance 4

- **objectif commun** : manipuler listes, dictionnaires et table CSV ;
- **Ahmad** : indexation, `len`, `append`, parcours ;
- **Ahmed** : aliasing, compréhensions, dictionnaires et validation des lignes ;
- **preuve attendue** : import CSV, filtre, tri et calcul statistique.

Séance 5

- **objectif commun** : résoudre un problème complet ;
- **Ahmad** : squelette guidé ;
- **Ahmed** : cahier des charges seul, tests supplémentaires et comparaison d'algorithmes ;
- **preuve attendue** : programme exécutable, documenté, testé et expliqué oralement.

6. Questions de relance

- « quelle est la valeur de cette variable avant la ligne ? »
- « que produit exactement `range` ? »
- « s'agit-il d'un compteur ou d'un accumulateur ? »
- « que renvoie la fonction ? »
- « quel test manque ? »
- « que se passe-t-il pour une liste vide ? »
- « le code modifie-t-il la liste d'origine ? »
- « quel est le coût quand la liste double ? »

7. Protocole de correction d'un bug

1. reproduire le bug ;
2. réduire l'exemple ;
3. identifier le premier état incorrect ;
4. formuler une hypothèse ;
5. ajouter un affichage temporaire ou une assertion ;
6. corriger une seule chose ;
7. relancer tout le jeu de tests ;
8. retirer les affichages de débogage inutiles.

8. Critères de fin de stage

- aucun mélange entre affectation et comparaison ;
- conditions et négations exactes ;
- boucles sans erreur de borne ;
- fonctions avec `return` adapté ;
- listes indexées à partir de zéro ;
- tests normaux et limites ;
- code organisé en fonctions ;
- import CSV et filtrage opérationnels ;
- présentation orale structurée.



PRÉ-RENTRÉE 2026-2027

Installer l'environnement

Python 3 et Thonny

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Guide famille et élève

Configuration recommandée

- un ordinateur portable par élève ;
- Python 3.11 ou supérieur ;
- Thonny récent ;
- 2 Go d'espace libre ;
- un dossier local de travail sauvegardé ;
- le pack de code élève correspondant, sans solution.

Installation

Windows et macOS

1. Télécharger Thonny depuis son site officiel.
2. Installer avec les options par défaut.
3. Ouvrir Thonny.
4. Vérifier dans **Outils > Options > Interpréteur** que Python 3 est sélectionné.
5. Exécuter :

```
print("Nexus NSI prêt")
```

Linux

Installer Python 3 et Thonny avec le gestionnaire de paquets de la distribution ou l'installateur officiel. Vérifier dans un terminal :

```
python3 --version
```

Packs de code à utiliser

Dans 09_PACKS_CODE/ :

- 1re_nsi_CODE_ELEVE_COMMUN.zip : fichiers communs sans solution ;
- 1re_nsi_CODE_Ahmad_BELDI.zip : fichiers et consignes du parcours Fondations ;
- 1re_nsi_CODE_Ahmed_BENHADJ_SALEM.zip : fichiers et consignes du parcours Reprise/ Examen ;
- 1re_nsi_CODE_ENSEIGNANT.zip : solutions et tests - à ne jamais copier sur un poste élève.

Arborescence conseillée

```
Premiere_NSI/  
├─ S1_variables_conditions/  
├─ S2_boucles/  
├─ S3_fonctions_tests/  
├─ S4_structures_csv/  
├─ S5_projet_capteurs/  
└─ sauvegardes/
```

Vérification de l'environnement

Créer test_installation.py :

```
from pathlib import Path  
  
print("Python fonctionne")  
print("Dossier courant :", Path.cwd())  
assert 2 + 2 == 4
```

Le programme doit s'exécuter sans erreur.

Règles de sauvegarde

- ne jamais écraser la dernière version qui fonctionne ;
- utiliser des noms comme `projet_v01.py`, `projet_v02.py` ;
- conserver `mesures_capteurs.csv` avec le projet ;
- enregistrer après chaque jalon ;
- ne jamais stocker de mot de passe, clé ou donnée personnelle dans un script ;
- ne pas utiliser de vraies données d'élèves.

En cas de problème

Noter le message exact, le système, la version de Python, le fichier et l'étape qui échoue. Joindre une capture du message complet ; ne pas se contenter de « ça ne marche pas ».



PRÉ-RENTRÉE 2026-2027

Matrice Python de Première NSI

Ce qu'il faut maîtriser pendant l'année

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Document enseignant et élève

Matrice de compétences

Domaine	Compétences Python	Exemples de preuves	Stage	À réactiver pendant l'année
Types de base	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code> , <code>None</code> , conversions	prédire <code>type</code> , éviter comparaison exacte de flottants	✓	✓
Affectation	état, évaluation droite puis stockage gauche	table de trace	✓	✓
Logique	comparaisons, <code>and</code> , <code>or</code> , <code>not</code> , négation	table de vérité et condition bornée	✓	✓
Conditions	<code>if</code> , <code>elif</code> , <code>else</code>	programme de classification	✓	✓
Boucles	<code>for</code> , <code>range</code> , <code>while</code>	compteur, accumulateur, variant	✓	✓
Fonctions	paramètres, arguments, <code>return</code> , portée, <code>None</code>	fonction documentée et testée	✓	✓
Contrats	préconditions, postconditions, assertions	<code>assert</code> sur cas normal et limite	✓	✓
Listes/ tableaux	index, modification, parcours, compréhension	transformer une liste	✓	✓
Tuples	regroupement de valeurs et retour multiple	fonction renvoyant (<code>min</code> , <code>max</code>)	✓	✓
Dictionnaires	clés, valeurs, <code>items</code>	enregistrement et compteur de fréquences	✓	✓
Tables CSV	<code>import</code> , sélection, tri, doublons, fusion	analyser un fichier de mesures	✓	✓
Algorithmes	recherche, extremum, moyenne, tri, dichotomie	code et trace	✓ partiel	✓
Coût	linéaire, quadratique, logarithmique	comparer nombre d'opérations	initiation	✓
Bibliothèques	lire une documentation, importer un module	<code>csv</code> , <code>math</code>	✓	✓
Projet	spécifier, découper, coder, tester, documenter	mini-projet final	✓	✓

Repères de qualité du code

- noms explicites ;
- fonction courte et cohérente ;
- pas de duplication inutile ;
- docstring concise ;
- préconditions explicites ;
- valeur renvoyée claire ;
- tests incluant cas limite ;
- aucun `print` de débogage laissé sans raison ;
- données réelles séparées du code ;
- commentaires sur le **pourquoi**, pas sur l'évidence syntaxique.



PRÉ-RENTRÉE 2026-2027

Tableau de bord enseignant

Suivi des cinq séances

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Ahmad BELDI - Ahmed BENHADJ SALEM

Échelle

0 non situé - 1 à reconstruire - 2 avec aide - 3 autonome - 4 transféré.

Grille récapitulative

Compétence	Ahmad initial	Ahmad final	Ahmed initial	Ahmed final	Preuve conservée
Affectation et trace	1		3		
Types et expressions	1		3		
Booléens et négations	2		2		
Conditions	2		3		
<code>for</code> et <code>range</code>	1		3		
<code>while</code> et terminaison	2		1		
Compteur/accumulateur	1		2		
Paramètres/arguments	1		3		
<code>return</code> / <code>None</code>	1		2		
Assertions et tests	0		1		
Indexation et <code>len</code>	1		2		
Listes et dictionnaires	1		3		
CSV et tables	2		3		
Recherche/extremum/moyenne	1		2		
Tri et dichotomie	0		2		
Documentation	0		2		
Autonomie	1		2		

Compétence	Ahmad initial	Ahmad final	Ahmed initial	Ahmed final	Preuve conservée
Explication orale	1		2		

Suivi par séance

Séance	Élève	Rituel	Atelier commun	Parcours	Aide max	Tests	Oral	Décision suivante
1	Ahmad			Fondations				
1	Ahmed			Fiabilisation				
2	Ahmad			Fondations				
2	Ahmed			Fiabilisation				
3	Ahmad			Fondations				
3	Ahmed			Fiabilisation				
4	Ahmad			Fondations				
4	Ahmed			Fiabilisation				
5	Ahmad			Projet guidé				
5	Ahmed			Projet autonome				

Notes de séance

Séance 1

Séance 2

Séance 3

Séance 4

Séance 5

Corrigé du test initial

Q	Réponse	Domaine	Lecture pédagogique
1	B	Binaire	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
2	D	Binaire	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
3	A	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
4	C	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
5	B	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
6	A	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
7	D	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
8	B	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
9	C	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
10	A	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
11	B	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
12	D	Programmation	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
13	A	Web	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
14	C	Web	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
15	B	Réseaux	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
16	A	Réseaux	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
17	C	Tables	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer
18	D	Métadonnées	Faux+sûr : confronter ; juste+hésitant : consolider ; vide : diagnostiquer

Questions d'entretien ciblées

- Ahmad : Q3, Q4, Q7, Q9, Q11 ou Q12.
- Ahmed : Q6, Q8, Q10, Q12, puis un accumulateur `s += i`.



PRÉ-RENTRÉE 2026-2027

Séance 1 - Fiche enseignant

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Corrigé, différenciation et pilotage

Objectifs

- comprendre l'ordre d'évaluation d'une affectation ;
- distinguer affectation et comparaison ;
- identifier les types de base ;
- écrire une condition et sa négation ;
- programmer un classificateur robuste ;

Déroulé minute par minute

- 0-10 installation et rituel ;
- 10-25 tables de trace ;
- 25-45 types et conversions ;
- 45-65 booléens et conditions ;
- 65-70 pause ;
- 70-95 atelier guidé ;
- 95-110 différenciation ;
- 110-118 tests et revue ;
- 118-120 exit ticket ;

Rituel prêt à l'emploi

Question 1

Tracer sans exécuter :

```
a = 4
b = a + 2
a = 10
```

Que valent `a` et `b` à la fin ?

Réponse attendue : `a` vaut 10 et `b` vaut 6 : l'affectation de `b` a copié la valeur 6.

Question 2

Écrire en Python la négation exacte de :

```
age >= 18
```

Réponse attendue : `age < 18`.

Notions de cours

État et affectation

Une variable est un nom associé à une valeur dans l'état courant du programme. Pour `x = expression`, Python :

1. évalue l'expression à droite ;
2. stocke la valeur obtenue sous le nom à gauche.

Ainsi, après `a = 3; b = a; a = 5`, `b` vaut encore 3.

Types de base

Type	Exemple	Usage
<code>int</code>	12, -4	entier
<code>float</code>	3.5	approximation d'un réel
<code>str</code>	"NSI"	texte
<code>bool</code>	True, False	condition
<code>NoneType</code>	None	absence de valeur

Booléens et conditions

- `and` exige que les deux conditions soient vraies ;
- `or` exige qu'au moins une condition soit vraie ;
- `not` nie la condition ;
- la négation de `x > 5` est `x <= 5`.

```

if temperature < 0:
    etat = "gel"
elif temperature <= 30:
    etat = "normal"
else:
    etat = "alerte"

```

Activité commune - tables de trace

A. Affectations

Compléter les états successifs.

```

a = 3
b = a
a = 5
c = a + b

```

Ligne exécutée	☐	☐	☐
départ	non défini	non défini	non défini
a = 3			
b = a			
a = 5			
c = a + b			

B. Conditions aux bornes

Pour chacune des valeurs -1, 0, 5, 10, 11, indiquer la valeur de :

$(x > 0) \text{ and } (x < 10)$

x	résultat	justification
-1		
0		
5		
10		

x	résultat	justification
11		

Parcours Fondations - Ahmad

1. Corriger le programme suivant :

```
age = "16"
print(age + 1)
```

2. Compléter pour afficher `pair`, `impair` ou `nul`.

```
n = -3
# À compléter
```

3. Écrire la négation Python de chacune des conditions :

- `x >= 0` ;
- `age < 18` ;
- `(x > 0) and (x < 10)`.

4. Écrire un programme qui classe une température en `gel`, `normal` ou `alerte`.

Parcours Fiabilisation - Ahmed

1. Écrire une condition vraie exactement lorsque `x` n'appartient pas à l'intervalle `[0, 10]`.
2. Simplifier la négation de `(x >= 2) and (x <= 8)`.
3. Corriger un programme comportant une branche inaccessible.
4. Écrire des tests sur les valeurs limites `-1`, `0`, `30`, `31` du classificateur de température.

Corrigé essentiel

- trace : `a=3`, puis `b=3`, puis `a=5`, enfin `c=8` ;
- la condition est vraie seulement pour `x=5` dans la liste proposée ;
- `age = int("16")` permet `age + 1` ;
- négation de `x >= 0` : `x < 0` ; de `age < 18` : `age >= 18` ;
- négation de `(x > 0) and (x < 10)` : `(x <= 0) or (x >= 10)`.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					



PRÉ-RENTRÉE 2026-2027

Séance 1 - Cahier élève

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ comprendre l'ordre d'évaluation d'une affectation ;
- ☐ distinguer affectation et comparaison ;
- ☐ identifier les types de base ;
- ☐ écrire une condition et sa négation ;
- ☐ programmer un classificateur robuste ;

Rituel sans ordinateur

Question 1

Tracer sans exécuter :

```
a = 4
b = a + 2
a = 10
```

Que valent `a` et `b` à la fin ?

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Écrire en Python la négation exacte de :

```
age >= 18
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

État et affectation

Une variable est un nom associé à une valeur dans l'état courant du programme. Pour `x = expression`, Python :

1. évalue l'expression à droite ;
2. stocke la valeur obtenue sous le nom à gauche.

Ainsi, après `a = 3` ; `b = a` ; `a = 5`, `b` vaut encore `3`.

Types de base

Type	Exemple	Usage
<code>int</code>	<code>12</code> , <code>-4</code>	entier
<code>float</code>	<code>3.5</code>	approximation d'un réel
<code>str</code>	<code>"NSI"</code>	texte
<code>bool</code>	<code>True</code> , <code>False</code>	condition
<code>NoneType</code>	<code>None</code>	absence de valeur

Booléens et conditions

- `and` exige que les deux conditions soient vraies ;
- `or` exige qu'au moins une condition soit vraie ;
- `not` nie la condition ;
- la négation de `x > 5` est `x <= 5`.

```
if temperature < 0:
    etat = "gel"
elif temperature <= 30:
    etat = "normal"
else:
    etat = "alerte"
```


Activité commune - tables de trace

A. Affectations

Compléter les états successifs.

```
a = 3
b = a
a = 5
c = a + b
```

Ligne exécutée			
départ	non défini	non défini	non défini
a = 3			
b = a			
a = 5			
c = a + b			

B. Conditions aux bornes

Pour chacune des valeurs -1, 0, 5, 10, 11, indiquer la valeur de :

$(x > 0) \text{ and } (x < 10)$

x	résultat	justification
-1		
0		
5		
10		
11		

Parcours Fondations - Ahmad

1. Corriger le programme suivant :

```
age = "16"  
print(age + 1)
```

2. Compléter pour afficher `pair`, `impair` ou `nul`.

```
n = -3  
# À compléter
```

3. Écrire la négation Python de chacune des conditions :

- `x >= 0` ;
- `age < 18` ;
- `(x > 0) and (x < 10)`.

4. Écrire un programme qui classe une température en `gel`, `normal` ou `alerte`.

Parcours Fiabilisation - Ahmed

1. Écrire une condition vraie exactement lorsque `x` n'appartient pas à l'intervalle `[0, 10]`.
2. Simplifier la négation de `(x >= 2) and (x <= 8)`.
3. Corriger un programme comportant une branche inaccessible.
4. Écrire des tests sur les valeurs limites `-1`, `0`, `30`, `31` du classificateur de température.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 1 - Supports pratiques

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - table mémoire

Découper ou projeter cette grille. Une ligne correspond à l'état **après** l'instruction.

Instruction	a	b	x	Type de x
départ				

Support 2 - cartes de types

int

Entier : 0, 12, -7

Conversion : `int(...)`

float

Nombre approché : 3.5

Conversion : `float(...)`

str

Texte : "NSI"

`input()` renvoie une chaîne.

bool

`True` ou `False`

Produit par une comparaison.

None

Absence de valeur.

Retour implicite d'une fonction sans `return`.

Support 3 - table de vérité

A	B	A and B	A or B	not A
False	False			
False	True			
True	False			
True	True			

Support 4 - cartes de bornes

-1

Juste avant la borne 0

0

Borne inférieure

10

Borne supérieure

11

Juste après la borne 10



PRÉ-RENTRÉE 2026-2027

Séance 1 - Cartes d'aide

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Lire

Lis une ligne à la fois et écris la valeur de chaque variable.

B - Type

Demande-toi si la valeur est un entier, un flottant, un texte ou un booléen.

C - Affecter

Évalue d'abord la partie droite, puis remplace la valeur associée au nom de gauche.

D - Condition

Teste séparément chaque comparaison avant de combiner avec and/or.

E - Vérifier

Teste les valeurs limites et trace le programme sans ordinateur.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 2 - Fiche enseignant

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Corrigé, différenciation et pilotage

Objectifs

- comprendre les valeurs produites par range ;
- choisir entre for et while ;
- utiliser compteur et accumulateur ;
- éviter les erreurs de borne ;
- justifier une terminaison simple ;

Déroulé minute par minute

- 0-10 rituel ;
- 10-30 range et bornes ;
- 30-50 compteurs/accumulateurs ;
- 50-65 for/while ;
- 65-70 pause ;
- 70-100 atelier ;
- 100-112 différenciation ;
- 112-120 invariant et exit ;

Rituel prêt à l'emploi

Question 1

Écrire, dans l'ordre, les valeurs produites par :

```
range(2, 9, 3)
```

Réponse attendue : 2, 5, 8.

Question 2

Tracer puis donner la valeur finale de `s` :


```
s = 0
for i in range(4):
    s = s + i
```

Réponse attendue : `s = 6`, car on ajoute $0 + 1 + 2 + 3$.

Notions de cours

Boucle bornée

```
for i in range(3):
    print(i)
```

produit `0`, `1`, `2`. La borne supérieure est exclue.

Boucle non bornée

```
while condition:
    # corps
```

Elle convient lorsque le nombre de répétitions n'est pas connu à l'avance. Il faut identifier un **variant** qui évolue vers l'arrêt.

Schémas de base

- compteur : `compteur += 1` ;
- accumulateur : `somme += valeur` ;
- recherche d'extremum : initialiser avec le premier élément, puis comparer ;
- parcours : traiter chaque élément exactement une fois.

Activité commune - prévoir avant d'exécuter

```
s = 0
for i in range(1, 4):
    s = s + i
```

Tour	☒ 1	☒ avant	☐ après
1			
2			

Tour	☐ 1	☐ avant	☐ après
3			

Choisir la boucle

Situation	for ou while ?	Justification
parcourir une liste		
demander un mot de passe jusqu'à réussite		
afficher 10 nombres		
lire des valeurs jusqu'au mot fin		

Parcours Fondations - Ahmad

1. Écrire les valeurs de `i` pour `range(5)`, `range(2, 6)` et `range(10, 3, -2)`.
2. Compléter un programme qui calcule la somme de 1 à `n`.
3. Compter le nombre de valeurs positives d'une liste.
4. Calculer le maximum sans utiliser `max`.
5. Corriger une boucle `while` qui ne se termine jamais.

Parcours Fiabilisation - Ahmed

1. Écrire une fonction de test manuel pour vérifier une somme cumulée.
2. Calculer simultanément somme, effectif, minimum et maximum en un seul parcours.
3. Traiter proprement la liste vide.
4. Expliquer un invariant du calcul de somme.
5. Écrire une boucle `while` dont le variant est explicite.

Corrigé essentiel

- `range(5)` : 0,1,2,3,4 ; `range(2, 6)` : 2,3,4,5 ; `range(10, 3, -2)` : 10,8,6,4 ;
- somme de 1 à `n` : initialiser `somme=0`, ajouter chaque entier ;
- maximum : initialiser avec le premier élément, puis remplacer si une valeur plus grande est rencontrée ;
- une boucle `while` doit modifier une donnée intervenant dans sa condition.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					



PRÉ-RENTRÉE 2026-2027

Séance 2 - Cahier élève

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ comprendre les valeurs produites par range ;
- ☐ choisir entre for et while ;
- ☐ utiliser compteur et accumulateur ;
- ☐ éviter les erreurs de borne ;
- ☐ justifier une terminaison simple ;

Rituel sans ordinateur

Question 1

Écrire, dans l'ordre, les valeurs produites par :

```
range(2, 9, 3)
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Tracer puis donner la valeur finale de `s` :

```
s = 0
for i in range(4):
    s = s + i
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

Boucle bornée

```
for i in range(3):  
    print(i)
```

produit 0, 1, 2. La borne supérieure est exclue.

Boucle non bornée

```
while condition:  
    # corps
```

Elle convient lorsque le nombre de répétitions n'est pas connu à l'avance. Il faut identifier un **variant** qui évolue vers l'arrêt.

Schémas de base

- compteur : `compteur += 1` ;
- accumulateur : `somme += valeur` ;
- recherche d'extremum : initialiser avec le premier élément, puis comparer ;
- parcours : traiter chaque élément exactement une fois.

Activité commune - prévoir avant d'exécuter

```
s = 0  
for i in range(1, 4):  
    s = s + i
```

Tour	☐	☐ avant	☐ après
1			
2			
3			

Choisir la boucle

Situation	for ou while ?	Justification
parcourir une liste		
demandeur un mot de passe jusqu'à réussite		
afficher 10 nombres		
lire des valeurs jusqu'au mot fin		

Parcours Fondations - Ahmad

1. Écrire les valeurs de `i` pour `range(5)`, `range(2, 6)` et `range(10, 3, -2)`.
2. Compléter un programme qui calcule la somme de `1` à `n`.
3. Compter le nombre de valeurs positives d'une liste.
4. Calculer le maximum sans utiliser `max`.
5. Corriger une boucle `while` qui ne se termine jamais.

Parcours Fiabilisation - Ahmed

1. Écrire une fonction de test manuel pour vérifier une somme cumulée.
2. Calculer simultanément somme, effectif, minimum et maximum en un seul parcours.
3. Traiter proprement la liste vide.
4. Expliquer un invariant du calcul de somme.
5. Écrire une boucle `while` dont le variant est explicite.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 2 - Supports pratiques

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - bandes range

Compléter avant toute exécution.

Expression	Première valeur	Dernière valeur produite	Nombre de tours	Valeurs
range(5)				
range(2, 8)				
range(10, 2, -2)				
range(5, 5)				

Support 2 - cartes de schémas de boucle

Compteur

Compte combien de fois une propriété est vraie.

```
compteur = 0  
...  
compteur += 1
```

Accumulateur

Construit progressivement une somme ou un résultat.

```
total = 0  
...  
total += valeur
```

Extremum

Part du premier élément puis compare les suivants.

Recherche

S'arrête ou renvoie un résultat dès que la cible est trouvée.

Support 3 - grille de trace de boucle

Tour	Valeur de boucle	État avant	Instruction	État après	Condition d'arrêt
1					
2					
3					
4					
5					

Support 4 - tri `for` / `while`

Découper puis classer : « parcourir une liste », « attendre un mot de passe valide », « répéter 20 fois », « lire jusqu'au mot fin », « parcourir les caractères d'un texte », « recommencer tant que la saisie est invalide ».



PRÉ-RENTRÉE 2026-2027

Séance 2 - Cartes d'aide

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Range

Écris explicitement les valeurs produites par range.

B - Schéma

Cherches-tu à compter, additionner, rechercher ou répéter jusqu'à une condition ?

C - Initialiser

Un compteur et une somme commencent souvent à 0 ; un maximum commence souvent au premier élément.

D - Boucle

Vérifie que la variable de boucle ou la condition évolue.

E - Vérifier

Compte les tours et compare le résultat à un calcul manuel.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 3 - Fiche enseignant

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Corrigé, différenciation et pilotage

Objectifs

- distinguer paramètre et argument ;
- distinguer print, return et None ;
- comprendre la portée locale ;
- documenter une fonction ;
- écrire des assertions de test ;

Déroulé minute par minute

- 0-10 rituel ;
- 10-30 paramètres/arguments ;
- 30-48 return/None ;
- 48-65 contrats ;
- 65-70 pause ;
- 70-100 module de fonctions ;
- 100-112 tests/débogage ;
- 112-120 revue/exit ;

Rituel prêt à l'emploi

Question 1

Que vaut `r` après l'exécution ?

```
def double(x):  
    print(2 * x)  
  
r = double(5)
```

Réponse attendue : Le programme affiche 10, mais `r` vaut `None` car la fonction ne contient pas de `return`.

Question 2

Dans `def aire_disque(rayon):`, distinguer le paramètre, l'argument dans `aire_disque(3)` et la valeur renvoyée.

Réponse attendue : Paramètre : `rayon` ; argument : `3` ; valeur renvoyée : celle placée après `return`.

Notions de cours

```
def carre(n: int) -> int:
    """Renvoie le carré de n."""
    return n * n
```

- `n` est un paramètre ;
- dans `carre(5)`, `5` est l'argument ;
- `return` termine la fonction et transmet une valeur ;
- `print` affiche mais ne renvoie pas cette valeur ;
- sans `return`, Python renvoie `None` ;
- les variables créées dans la fonction sont locales.

Contrat

- précondition : ce qui doit être vrai avant l'appel ;
- postcondition : ce qui est garanti après l'appel ;
- `assert` peut vérifier une condition ;
- un jeu de tests ne prouve pas l'absence de bugs, mais il réduit le risque.

Activité commune - prédire les retours

```
def f(x):
    print(x + 1)

resultat = f(4)
```

1. Qu'est-ce qui est affiché ?
2. Que vaut `resultat` ?
3. Comment modifier `f` pour renvoyer 5 ?

Conception de tests

Pour une fonction `maximum_deux(a, b)`, proposer :

- un cas ordinaire ;
- un cas où `a == b` ;
- un cas avec deux valeurs négatives.

Parcours Fondations - Ahmad

1. Écrire `est_pair(n)`.
2. Écrire `maximum_deux(a, b)`.
3. Écrire `compte_occurrences(valeurs, cible)`.
4. Ajouter une docstring à chaque fonction.
5. Écrire deux assertions par fonction.

Parcours Fiabilisation - Ahmed

1. Écrire une précondition pour une fonction de moyenne.
2. Choisir le comportement sur liste vide : assertion ou `None`, et le justifier.
3. Écrire des tests limites et invalides.
4. Corriger une fonction qui modifie une variable globale.
5. Lire la documentation de `math.isclose` et l'utiliser pour tester un flottant.

Corrigé essentiel

- le programme affiche `5` et `resultat` vaut `None` ;
- pour renvoyer `5` : `return x + 1` ;
- `est_pair(n)` renvoie `n % 2 == 0` ;
- les tests doivent couvrir égalité, négatifs, zéro et cas limites ;
- une fonction de moyenne doit préciser le comportement sur une liste vide.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					



PRÉ-RENTRÉE 2026-2027

Séance 3 - Cahier élève

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ distinguer paramètre et argument ;
- ☐ distinguer print, return et None ;
- ☐ comprendre la portée locale ;
- ☐ documenter une fonction ;
- ☐ écrire des assertions de test ;

Rituel sans ordinateur

Question 1

Que vaut `r` après l'exécution ?

```
def double(x):  
    print(2 * x)  
  
r = double(5)
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Dans `def aire_disque(rayon):`, distinguer le paramètre, l'argument dans `aire_disque(3)` et la valeur renvoyée.

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

```
def carre(n: int) -> int:
    """Renvoie le carré de n."""
    return n * n
```

- `n` est un paramètre ;
- dans `carre(5)`, `5` est l'argument ;
- `return` termine la fonction et transmet une valeur ;
- `print` affiche mais ne renvoie pas cette valeur ;
- sans `return`, Python renvoie `None` ;
- les variables créées dans la fonction sont locales.

Contrat

- précondition : ce qui doit être vrai avant l'appel ;
- postcondition : ce qui est garanti après l'appel ;
- `assert` peut vérifier une condition ;
- un jeu de tests ne prouve pas l'absence de bugs, mais il réduit le risque.

Activité commune - prédire les retours

```
def f(x):
    print(x + 1)

resultat = f(4)
```

1. Qu'est-ce qui est affiché ?
2. Que vaut `resultat` ?
3. Comment modifier `f` pour renvoyer 5 ?

Conception de tests

Pour une fonction `maximum_deux(a, b)`, proposer :

- un cas ordinaire ;
- un cas où `a == b` ;
- un cas avec deux valeurs négatives.

Parcours Fondations - Ahmad

1. Écrire `est_pair(n)`.
2. Écrire `maximum_deux(a, b)`.
3. Écrire `compte_occurrences(valeurs, cible)`.
4. Ajouter une docstring à chaque fonction.
5. Écrire deux assertions par fonction.

Parcours Fiabilisation - Ahmed

1. Écrire une précondition pour une fonction de moyenne.
2. Choisir le comportement sur liste vide : assertion ou `None`, et le justifier.
3. Écrire des tests limites et invalides.
4. Corriger une fonction qui modifie une variable globale.
5. Lire la documentation de `math.isclose` et l'utiliser pour tester un flottant.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 3 - Supports pratiques

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - anatomie d'une fonction

```
def nom_fonction(parametre_1, parametre_2):  
    """Description brève du résultat et des préconditions."""  
    # calcul local  
    return resultat
```

Élément	Rôle	Exemple personnel
nom	intention	
paramètre	entrée symbolique	
argument	valeur lors de l'appel	
variable locale	état interne	
return	sortie transmise	

Support 2 - cartes « afficher ou renvoyer ? »

print

Affiche pour un humain.

La valeur affichée n'est pas automatiquement réutilisable.

return

Transmet une valeur à l'appelant.

Met fin à l'exécution de la fonction.

None

Valeur obtenue si aucun `return` explicite n'est exécuté.

assert

Vérifie une condition du contrat ou un résultat de test.

Support 3 - fabrique de tests

Fonction	Cas normal	Cas limite	Cas égalité	Cas invalide	Résultat attendu

Support 4 - ticket de débogage

1. Symptôme observé :
2. Plus petit exemple qui échoue :
3. Premier état incorrect :
4. Hypothèse :
5. Test après correction :



PRÉ-RENTRÉE 2026-2027

Séance 3 - Cartes d'aide

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Interface

Liste les paramètres et la valeur attendue en sortie.

B - Return

print affiche ; return transmet une valeur à l'appelant.

C - Portée

Une variable créée dans une fonction est locale sauf indication explicite.

D - Contrat

Écris une précondition et une postcondition.

E - Tests

Ajoute un cas normal, un cas limite et un cas invalide.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 4 - Fiche enseignant

Listes, tuples, dictionnaires et tables CSV

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Corrigé, différenciation et pilotage

Objectifs

- maîtriser indexation et longueur ;
- comprendre mutation et aliasing ;
- utiliser listes et dictionnaires ;
- importer une table CSV ;
- filtrer, trier et contrôler des données ;

Déroulé minute par minute

- 0-10 rituel ;
- 10-28 listes/index ;
- 28-45 aliasing ;
- 45-65 dictionnaires ;
- 65-70 pause ;
- 70-100 CSV ;
- 100-112 tri/cohérence ;
- 112-120 synthèse/exit ;

Rituel prêt à l'emploi

Question 1

Pour `L = [4, 8, 15, 16]`, donner `L[1]`, `len(L)` et le dernier indice.

Réponse attendue : `L[1] = 8`, `len(L) = 4`, dernier indice : 3.

Question 2

Prédire la valeur de `a` :

```
a = [1, 2]
b = a
b.append(3)
```

Réponse attendue : `a` vaut `[1, 2, 3]`, car `a` et `b` sont deux noms du même objet liste.

Notions de cours

Listes

- premier élément : `L[0]` ;
- dernier élément : `L[-1]` ;
- longueur : `len(L)` ;
- ajout : `L.append(x)` ;
- parcours : `for valeur in L:` ;
- compréhension : `[x*x for x in L if x >= 0]`.

Alias

```
a = [1, 2]
b = a
b.append(3)
```

`a` et `b` désignent la même liste. Pour une copie superficielle : `b = a.copy()`.

Dictionnaires

```
mesure = {"id": "C01", "temperature": 28.4}
```

Parcours : `for cle, valeur in mesure.items():`.

CSV

La bibliothèque standard `csv` permet d'importer une table. Chaque ligne peut devenir un dictionnaire avec `csv.DictReader`.

Activité commune - index, longueur et mutation

```
L = [4, 8, 15, 16]
```

Expression	Valeur
<code>L[0]</code>	
<code>L[1]</code>	
<code>len(L)</code>	
<code>L[len(L)-1]</code>	

Alias

Prédire puis exécuter :

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Expliquer le résultat.

Parcours Fondations - Ahmad

1. Corriger cinq expressions d'indexation.
2. Écrire une boucle qui double chaque valeur sans modifier la liste d'origine.
3. Construire un dictionnaire décrivant une mesure.
4. Parcourir `keys`, `values` et `items`.
5. Importer `mesures_capteurs.csv` et afficher les identifiants.

Parcours Fiabilisation - Ahmed

1. Démontrer un effet d'aliasing et le corriger.
2. Construire une compréhension filtrant les alertes.
3. Valider chaque ligne du CSV.
4. Détecter un identifiant dupliqué.
5. Trier les mesures par température puis par identifiant.

Corrigé essentiel

- `L[0]=4`, `L[1]=8`, `len(L)=4`, dernier élément `16` ;
- l'aliasing modifie les deux noms car ils désignent le même objet ;
- une copie indépendante se crée avec `copy()` ;
- `csv.DictReader` produit des dictionnaires dont les valeurs sont d'abord des chaînes : il faut convertir les nombres.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					



PRÉ-RENTRÉE 2026-2027

Séance 4 - Cahier élève

Listes, tuples, dictionnaires et tables CSV

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ maîtriser indexation et longueur ;
- ☐ comprendre mutation et aliasing ;
- ☐ utiliser listes et dictionnaires ;
- ☐ importer une table CSV ;
- ☐ filtrer, trier et contrôler des données ;

Rituel sans ordinateur

Question 1

Pour `L = [4, 8, 15, 16]`, donner `L[1]`, `len(L)` et le dernier indice.

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Prédire la valeur de `a` :

```
a = [1, 2]
b = a
b.append(3)
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

Listes

- premier élément : `L[0]` ;
- dernier élément : `L[-1]` ;
- longueur : `len(L)` ;
- ajout : `L.append(x)` ;
- parcours : `for valeur in L:` ;
- compréhension : `[x*x for x in L if x >= 0]`.

Alias

```
a = [1, 2]
b = a
b.append(3)
```

`a` et `b` désignent la même liste. Pour une copie superficielle : `b = a.copy()`.

Dictionnaires

```
mesure = {"id": "C01", "temperature": 28.4}
```

Parcours : `for cle, valeur in mesure.items():`.

CSV

La bibliothèque standard `csv` permet d'importer une table. Chaque ligne peut devenir un dictionnaire avec `csv.DictReader`.

Activité commune - index, longueur et mutation

```
L = [4, 8, 15, 16]
```

Expression	Valeur
<code>L[0]</code>	
<code>L[1]</code>	
<code>len(L)</code>	
<code>L[len(L)-1]</code>	

Alias

Prédire puis exécuter :

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Expliquer le résultat.

Parcours Fondations - Ahmad

1. Corriger cinq expressions d'indexation.
2. Écrire une boucle qui double chaque valeur sans modifier la liste d'origine.
3. Construire un dictionnaire décrivant une mesure.
4. Parcourir `keys`, `values` et `items`.
5. Importer `mesures_capteurs.csv` et afficher les identifiants.

Parcours Fiabilisation - Ahmed

1. Démontrer un effet d'aliasing et le corriger.
2. Construire une compréhension filtrant les alertes.
3. Valider chaque ligne du CSV.
4. Détecter un identifiant dupliqué.
5. Trier les mesures par température puis par identifiant.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 4 - Supports pratiques

Listes, tuples, dictionnaires et tables CSV

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - bande d'indices

Pour $L = [4, 8, 15, 16]$:

Indice	0	1	2	3
Valeur	4	8	15	16

- longueur : 4 ;
- dernier indice : 3 ;
- dernier élément : $L[-1]$.

Support 2 - alias ou copie ?

Alias

```
a = [1, 2]
b = a
```

Deux noms, un seul objet.

Copie superficielle

```
a = [1, 2]
b = a.copy()
```

Deux listes distinctes au premier niveau.

Support 3 - fiche enregistrement

Clé	Valeur	Type attendu	Valide ?
id		str non vide	
date		str ISO	
temperature		float	

Clé	Valeur	Type attendu	Valide ?
humidite		int	
statut		OK ou ALERTE	

Support 4 - mini-table CSV

```
id,date,temperature,humidite,statut
C01,2026-08-24,28.4,55,OK
C02,2026-08-24,31.2,52,ALERTE
C03,2026-08-24,-1.5,80,ALERTE
```

Questions : quelles valeurs doivent être converties ? Quelles contraintes vérifier ? Comment détecter un identifiant dupliqué ?



PRÉ-RENTRÉE 2026-2027

Séance 4 - Cartes d'aide

Listes, tuples, dictionnaires et tables CSV

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Indice

Le premier indice est 0 et le dernier $\text{len}(L)-1$.

B - Mutation

Demande-toi si l'opération modifie la liste ou construit une nouvelle liste.

C - Dictionnaire

Accède à une valeur avec sa clé et parcours avec `items()`.

D - CSV

Les valeurs lues sont des chaînes : convertis int/float avant de calculer.

E - Cohérence

Cherche valeurs manquantes, types invalides et identifiants dupliqués.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 5 - Fiche enseignant

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Corrigé, différenciation et pilotage

Objectifs

- écrire une recherche séquentielle ;
- calculer extremum et moyenne ;
- comprendre un tri par insertion ;
- tracer une recherche dichotomique ;
- livrer un projet documenté et testé ;

Déroulé minute par minute

- 0-10 quiz ;
- 10-27 recherche/extremum ;
- 27-43 insertion ;
- 43-58 dichotomie/coût ;
- 58-63 brief projet ;
- 63-68 pause ;
- 68-105 projet ;
- 105-115 oral/tests ;
- 115-120 bilan ;

Rituel prêt à l'emploi

Question 1

Dans une liste non triée, quel algorithme simple permet de trouver la première occurrence d'une cible ? Que renvoie-t-on si elle est absente ?

Réponse attendue : Une recherche séquentielle ; on peut renvoyer `None` si la cible est absente.

Question 2

Quel prérequis est indispensable à la recherche dichotomique et que devient approximativement la zone de recherche à chaque étape ?

Réponse attendue : La liste doit être triée ; la zone restante est divisée approximativement par deux.

Notions de cours

Recherche séquentielle

Parcourt les éléments un à un. Coût proportionnel au nombre d'éléments.

Tri par insertion

Maintient une partie gauche déjà triée et insère l'élément courant à sa place. Le pire cas est quadratique.

Recherche dichotomique

Dans une liste triée, compare à l'élément central et élimine la moitié restante. Le nombre d'étapes croît comme un logarithme.

Correction et terminaison

- invariant : propriété vraie avant et après chaque tour ;
- variant : entier positif qui décroît vers zéro ;
- tests : cas présent, absent, premier, dernier, liste vide.

Activité commune - choisir l'algorithme

Besoin	Algorithme	Précondition	Coût intuitif
trouver un identifiant dans une liste quelconque			
trouver dans une liste triée très longue			
remettre une petite liste dans l'ordre			
trouver la plus grande température			

Projet final

À partir de `mesures_capteurs.csv`, produire un programme qui :

1. charge et valide les données ;
2. calcule moyenne, minimum et maximum ;
3. filtre les alertes ;
4. recherche un capteur par identifiant ;
5. trie les mesures ;
6. affiche un rapport lisible ;
7. contient des fonctions, docstrings et assertions.

Parcours Projet guidé - Ahmad

- utiliser le fichier `projet_final_starter_ahmad.py` ;
- compléter les fonctions dans l'ordre indiqué ;
- exécuter les tests fournis après chaque fonction ;
- expliquer deux fonctions lors de la restitution.

Parcours Projet autonome - Ahmed

- utiliser `projet_final_specification_ahmed.md` ;
- définir l'architecture du programme avant de coder ;
- écrire les tests, y compris liste vide et identifiant absent ;
- comparer recherche séquentielle et dichotomique ;
- expliquer la complexité intuitive et un invariant.

Corrigé essentiel

Le pack `09_PACKS_CODE/1re_nsi_CODE_ENSEIGNANT.zip` contient la solution de référence.
Une autre solution est recevable si elle respecte le contrat, les tests et la lisibilité.

Critères : exactitude, décomposition, tests, documentation, absence de duplication, gestion des cas limites et explication orale.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					



PRÉ-RENTRÉE 2026-2027

Séance 5 - Cahier élève

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ écrire une recherche séquentielle ;
- ☐ calculer extremum et moyenne ;
- ☐ comprendre un tri par insertion ;
- ☐ tracer une recherche dichotomique ;
- ☐ livrer un projet documenté et testé ;

Rituel sans ordinateur

Question 1

Dans une liste non triée, quel algorithme simple permet de trouver la première occurrence d'une cible ? Que renvoie-t-on si elle est absente ?

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Quel prérequis est indispensable à la recherche dichotomique et que devient approximativement la zone de recherche à chaque étape ?

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

Recherche séquentielle

Parcourt les éléments un à un. Coût proportionnel au nombre d'éléments.

Tri par insertion

Maintient une partie gauche déjà triée et insère l'élément courant à sa place. Le pire cas est quadratique.

Recherche dichotomique

Dans une liste triée, compare à l'élément central et élimine la moitié restante. Le nombre d'étapes croît comme un logarithme.

Correction et terminaison

- invariant : propriété vraie avant et après chaque tour ;
- variant : entier positif qui décroît vers zéro ;
- tests : cas présent, absent, premier, dernier, liste vide.

Activité commune - choisir l'algorithme

Besoin	Algorithme	Précondition	Coût intuitif
trouver un identifiant dans une liste quelconque			
trouver dans une liste triée très longue			
remettre une petite liste dans l'ordre			
trouver la plus grande température			

Projet final

À partir de `mesures_capteurs.csv`, produire un programme qui :

1. charge et valide les données ;
2. calcule moyenne, minimum et maximum ;
3. filtre les alertes ;
4. recherche un capteur par identifiant ;
5. trie les mesures ;
6. affiche un rapport lisible ;
7. contient des fonctions, docstrings et assertions.

Parcours Projet guidé - Ahmad

- utiliser le fichier `projet_final_starter_ahmad.py` ;
- compléter les fonctions dans l'ordre indiqué ;
- exécuter les tests fournis après chaque fonction ;
- expliquer deux fonctions lors de la restitution.

Parcours Projet autonome - Ahmed

- utiliser `projet_final_specification_ahmed.md` ;
- définir l'architecture du programme avant de coder ;
- écrire les tests, y compris liste vide et identifiant absent ;
- comparer recherche séquentielle et dichotomique ;
- expliquer la complexité intuitive et un invariant.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 5 - Supports pratiques

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - cartes d'algorithmes

Recherche séquentielle

Liste quelconque. Examine au pire tous les éléments.

Recherche dichotomique

Liste triée. Élimine environ la moitié de la zone à chaque étape.

Tri par insertion

Maintient une partie gauche triée et insère l'élément courant.

Extremum

Un parcours suffit pour trouver minimum ou maximum.

Support 2 - choix et coût

Besoin	Données triées ?	Algorithme	Nombre d'éléments doublé : effet attendu
trouver une cible	non		
trouver une cible	oui		
trier une petite liste	non		
trouver un maximum	sans importance		

Support 3 - tableau de pilotage du projet

Fonction	Spécification	Écrite	Test normal	Test limite	Validée
charger		☐	☐	☐	☐
valider		☐	☐	☐	☐
statistiques		☐	☐	☐	☐
filtrer		☐	☐	☐	☐
rechercher		☐	☐	☐	☐
trier		☐	☐	☐	☐
rapport		☐	☐	☐	☐

Support 4 - grille d'oral de deux minutes

- problème traité ;
- données d'entrée ;
- décomposition en fonctions ;
- un test important ;
- un bug rencontré ;
- résultat obtenu ;
- amélioration possible.



PRÉ-RENTRÉE 2026-2027

Séance 5 - Cartes d'aide

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Spécifier

Écris les entrées, sorties et cas limites avant le code.

B - Décomposer

Une fonction par responsabilité : charger, valider, calculer, filtrer, rechercher, afficher.

C - Algorithme

Choisis recherche séquentielle ou dichotomique selon que la liste est triée.

D - Tests

Teste présent, absent, premier, dernier et vide.

E - Expliquer

Prépare une phrase sur la correction, la terminaison et le coût.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Corrigé du mini-diagnostic

Python - pilotage enseignant

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document enseignant

Réponses

1. `a = 10`, `b = 5`.
2. `range(2,6)` produit 2,3,4,5 ; `s = 14`.
3. `L[1] = 8`, `len(L)=3`, dernier indice 2.
4. le programme affiche 6 ; `r` vaut `None`.

```
def compte_superieurs(valeurs, seuil):  
    compteur = 0  
    for valeur in valeurs:  
        if valeur > seuil:  
            compteur += 1  
    return compteur  
  
assert compte_superieurs([1, 5, 8], 4) == 2  
assert compte_superieurs([], 4) == 0
```

Lecture pédagogique

Élément	Indice d'une difficulté
<code>b</code> suit encore <code>a</code>	modèle d'affectation erroné
somme 4 ou nombre de tours	accumulateur non compris
<code>L[1]=5</code>	indexation à partir de 1
<code>r=6</code>	confusion <code>print</code> / <code>return</code>
compteur réinitialisé dans la boucle	état de boucle non compris



PRÉ-RENTRÉE 2026-2027

Corrigé et barème

Évaluation finale Python

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Document enseignant

Partie A

1. `=` affecte ; `==` compare.
2. `(x < 0) or (x >= 10)`.
3. `5, 3, 1`.
4. `len(L)` est le nombre d'éléments ; le dernier indice est `len(L) - 1`.
5. `None`.
6. Les flottants sont représentés de façon approximative ; utiliser une tolérance.
7. La collection doit être triée selon la clé recherchée.
8. La liste vide.

Partie B - solution de référence

```
def valeurs_valides(valeurs, minimum, maximum):
    assert minimum <= maximum
    resultat = []
    for valeur in valeurs:
        if minimum <= valeur <= maximum:
            resultat.append(valeur)
    return resultat

def statistiques(valeurs):
    assert len(valeurs) > 0
    plus_petite = valeurs[0]
    plus_grande = valeurs[0]
    total = 0
    for valeur in valeurs:
        if valeur < plus_petite:
            plus_petite = valeur
        if valeur > plus_grande:
            plus_grande = valeur
        total += valeur
    return plus_petite, plus_grande, total / len(valeurs)

def indice_capteur(mesures, identifiant):
    for i in range(len(mesures)):
        if mesures[i]["id"] == identifiant:
            return i
    return None
```

Barème - 20 points

Critère	Points
connaissances et traçage	8
exactitude des fonctions	5
décomposition et lisibilité	2
tests	2
cas limites et contrats	1
explication orale	2

Le résultat n'est pas utilisé comme note de classement. Une erreur avec forte certitude déclenche une remédiation prioritaire.

QCM blanc - Candidat individuel Première NSI

Durée : 2 heures - 42 questions - 7 parties - calculatrice interdite.

Une seule réponse est correcte. Le format d'entraînement reprend l'évaluation ponctuelle officielle : 1 point par bonne réponse, 0 sinon, puis conversion sur 20 par $\text{points} \times 20 / 42$.

Différence avec le test de positionnement Nexus : dans ce QCM blanc, une réponse fausse n'enlève pas de point. Après une première passe prudente, il est rationnel de répondre à toutes les questions.

Partie - Données de base

Question 1

101101 en binaire vaut en décimal :

- ☐ A. 35
- ☐ B. 45
- ☐ C. 46
- ☐ D. 53

Question 2

Sur 8 bits non signés, la plus grande valeur est :

- ☐ A. 127
- ☐ B. 128
- ☐ C. 255
- ☐ D. 256

Question 3

En complément à 2 sur 8 bits, -1 est représenté par :

- ☐ A. 00000001
- ☐ B. 10000001
- ☐ C. 11111111
- ☐ D. 10000000

Question 4

En Python, l'affirmation correcte est :

- ☐ A. $0.1+0.2$ vaut toujours exactement 0.3
- ☐ B. les flottants sont des rationnels exacts
- ☐ C. il faut éviter de tester directement certaines égalités de flottants
- ☐ D. float et int ont toujours la même représentation

Question 5

`True and not False` vaut :

- ☐ A. True
- ☐ B. False
- ☐ C. None
- ☐ D. 1.0

Question 6

Unicode sert principalement à :

- ☐ A. compresser les images
- ☐ B. identifier des caractères de nombreuses écritures
- ☐ C. chiffrer les fichiers
- ☐ D. router des paquets

Partie - Types construits

Question 7

Un tuple Python est :

- ☐ A. toujours modifiable
- ☐ B. un regroupement ordonné non mutable
- ☐ C. un dictionnaire
- ☐ D. un fichier CSV

Question 8

Pour `L=[10,20,30]` , `L[2]` vaut :

- ☐ A. 20
- ☐ B. 30
- ☐ C. IndexError
- ☐ D. 3

Question 9

`[x*x for x in range(4)]` vaut :

- ☐ A. [1,4,9,16]
- ☐ B. [0,1,4,9]
- ☐ C. [0,1,2,3]
- ☐ D. [4]

Question 10

Pour `M=[[1,2],[3,4]]` , `M[1][0]` vaut :

- ☐ A. 1
- ☐ B. 2
- ☐ C. 3
- ☐ D. 4

Question 11

Pour `d={"a":1}`, ajouter la clé `b` associée à 2 se fait par :

- ☐ A. `d.add("b",2)`
- ☐ B. `d["b"]=2`
- ☐ C. `d.append(2)`
- ☐ D. `d("b")=2`

Question 12

Après `a=[1,2]` ; `b=a` ; `b.append(3)`, `a` vaut :

- ☐ A. `[1,2]`
- ☐ B. `[1,2,3]`
- ☐ C. `[3]`
- ☐ D. erreur

Partie - Tables

Question 13

Dans une table CSV, une ligne correspond généralement à :

- ☐ A. un descripteur
- ☐ B. un enregistrement
- ☐ C. un fichier entier
- ☐ D. un type Python

Question 14

`csv.DictReader` produit chaque ligne sous forme de :

- ☐ A. chaîne unique
- ☐ B. liste d'entiers
- ☐ C. dictionnaire
- ☐ D. tuple de bytes

Question 15

Pour garder les lignes de score ≥ 10 , on effectue :

- ☐ A. une projection
- ☐ B. un filtrage
- ☐ C. une fusion
- ☐ D. un encodage

Question 16

Trier une liste de dictionnaires `t` par score peut s'écrire :

- ☐ A. `sorted(t, key=lambda x: x["score"])`
- ☐ B. `t.sort("score")`
- ☐ C. `sort(score,t)`
- ☐ D. `sorted("score")`

Question 17

Fusionner deux tables suppose notamment :

- ☐ A. aucun champ commun
- ☐ B. un critère de correspondance
- ☐ C. qu'elles aient la même longueur
- ☐ D. qu'elles soient triées en binaire

Question 18

Une métadonnée est :

- ☐ A. une donnée fausse
- ☐ B. une donnée qui décrit une autre donnée
- ☐ C. une donnée secrète
- ☐ D. une copie de sauvegarde

Partie - Web

Question 19

Le rôle principal de HTML est de :

- ☐ A. mettre en forme
- ☐ B. structurer le contenu
- ☐ C. router les paquets
- ☐ D. stocker les utilisateurs

Question 20

Le rôle principal de CSS est de :

- ☐ A. présenter le contenu
- ☐ B. exécuter Python
- ☐ C. définir les adresses IP
- ☐ D. gérer le disque

Question 21

Dans le modèle client-serveur :

- ☐ A. le serveur interroge toujours le client
- ☐ B. le client envoie une requête et reçoit une réponse
- ☐ C. aucun protocole n'est utilisé
- ☐ D. le navigateur fabrique tout sans réseau

Question 22

Une requête GET est adaptée lorsque :

- ☐ A. on transmet un mot de passe en clair dans l'URL
- ☐ B. on demande une ressource sans effet sensible
- ☐ C. on veut garantir la confidentialité
- ☐ D. on remplace HTTPS

Question 23

HTTPS apporte principalement :

- ☐ A. la compression
- ☐ B. le chiffrement et l'authentification du serveur
- ☐ C. un stockage local
- ☐ D. une adresse IP fixe

Question 24

Un gestionnaire d'événement est exécuté :

- ☐ A. lorsqu'un événement associé se produit
- ☐ B. uniquement au démarrage du serveur
- ☐ C. avant le chargement de la page dans tous les cas
- ☐ D. par le routeur

Partie - Architecture

Question 25

Dans une architecture de von Neumann, le processeur :

- ☐ A. stocke uniquement les images
- ☐ B. exécute les instructions
- ☐ C. remplace le système d'exploitation
- ☐ D. attribue les adresses IP

Question 26

Le système d'exploitation :

- ☐ A. gère ressources, fichiers et processus
- ☐ B. est une page Web
- ☐ C. est un protocole de routage
- ☐ D. remplace la mémoire vive

Question 27

Une adresse IP sert à :

- ☐ A. identifier une machine sur un réseau
- ☐ B. nommer un fichier
- ☐ C. compresser un paquet
- ☐ D. chiffrer une chaîne

Question 28

Un routeur :

- ☐ A. exécute les fonctions Python
- ☐ B. transmet les paquets vers leur destination
- ☐ C. convertit toujours HTML en CSS
- ☐ D. stocke uniquement les mots de passe

Question 29

Le découpage en paquets permet notamment :

- ☐ A. d'interdire tout partage du réseau
- ☐ B. de transmettre et réassembler des données
- ☐ C. de supprimer les protocoles
- ☐ D. de garantir zéro perte sans mécanisme

Question 30

Sous Unix, la commande `ls` sert à :

- ☐ A. lister les fichiers
- ☐ B. changer de répertoire
- ☐ C. supprimer un fichier
- ☐ D. afficher une adresse IP

Partie - Programmation

Question 31

Après `a=3` ; `b=a` ; `a=5` , b vaut :

- ☐ A. 3
- ☐ B. 5
- ☐ C. 8
- ☐ D. None

Question 32

La négation de `x > 5` est :

- ☐ A. `x < 5`
- ☐ B. `x >= 5`
- ☐ C. `x <= 5`
- ☐ D. `x != 5`

Question 33

`range(2,8,2)` produit :

- ☐ A. 2,4,6
- ☐ B. 2,4,6,8
- ☐ C. 2,3,4,5,6,7
- ☐ D. 8,6,4,2

Question 34

Une boucle while est particulièrement adaptée lorsque :

- ☐ A. le nombre de tours dépend d'une condition
- ☐ B. le nombre de tours est toujours exactement 3
- ☐ C. on veut définir un dictionnaire
- ☐ D. on écrit une page HTML

Question 35

Une fonction sans return exécuté renvoie :

- ☐ A. 0
- ☐ B. False
- ☐ C. None
- ☐ D. une erreur systématique

Question 36

Un bon jeu de tests :

- ☐ A. prouve toujours la correction
- ☐ B. comprend cas normaux et limites
- ☐ C. ne contient qu'un exemple
- ☐ D. remplace la spécification

Partie - Algorithmique

Question 37

La recherche séquentielle :

- ☐ A. examine des éléments un à un
- ☐ B. exige une liste triée
- ☐ C. est toujours logarithmique
- ☐ D. trie la liste

Question 38

Pour calculer un maximum dans une liste non vide, une bonne initialisation est :

- ☐ A. 0 dans tous les cas
- ☐ B. le premier élément
- ☐ C. None sans traitement
- ☐ D. la longueur de la liste

Question 39

Le tri par sélection place à chaque étape :

- ☐ A. un élément choisi au hasard
- ☐ B. le plus petit élément restant à sa position
- ☐ C. la moitié droite à gauche
- ☐ D. la moyenne en tête

Question 40

Dans le pire cas, les tris insertion et sélection étudiés sont de coût :

- ☐ A. constant
- ☐ B. logarithmique
- ☐ C. linéaire
- ☐ D. quadratique

Question 41

La recherche dichotomique nécessite :

- ☐ A. une liste triée
- ☐ B. un dictionnaire vide
- ☐ C. un réseau
- ☐ D. une fonction récursive obligatoirement

Question 42

Une stratégie gloutonne :

- ☐ A. explore forcément toutes les solutions
- ☐ B. fait un choix local à chaque étape
- ☐ C. garantit toujours l'optimum pour tout problème
- ☐ D. est identique à la dichotomie

Corrigé du QCM candidat individuel

Q	Partie	Réponse
1	Données de base	B - 45
2	Données de base	C - 255
3	Données de base	C - 11111111
4	Données de base	C - il faut éviter de tester directement certaines égalités de flottants
5	Données de base	A - True
6	Données de base	B - identifier des caractères de nombreuses écritures
7	Types construits	B - un regroupement ordonné non mutable
8	Types construits	B - 30
9	Types construits	B - [0,1,4,9]
10	Types construits	C - 3
11	Types construits	B - d["b"]=2
12	Types construits	B - [1,2,3]
13	Tables	B - un enregistrement
14	Tables	C - dictionnaire
15	Tables	B - un filtrage
16	Tables	A - sorted(t, key=lambda x: x["score"])
17	Tables	B - un critère de correspondance
18	Tables	B - une donnée qui décrit une autre donnée
19	Web	B - structurer le contenu
20	Web	A - présenter le contenu
21	Web	B - le client envoie une requête et reçoit une réponse
22	Web	B - on demande une ressource sans effet sensible
23	Web	B - le chiffrement et l'authentification du serveur

Q	Partie	Réponse
24	Web	A - lorsqu'un événement associé se produit
25	Architecture	B - exécute les instructions
26	Architecture	A - gère ressources, fichiers et processus
27	Architecture	A - identifier une machine sur un réseau
28	Architecture	B - transmet les paquets vers leur destination
29	Architecture	B - de transmettre et réassembler des données
30	Architecture	A - lister les fichiers
31	Programmation	A - 3
32	Programmation	C - $x \leq 5$
33	Programmation	A - 2,4,6
34	Programmation	A - le nombre de tours dépend d'une condition
35	Programmation	C - None
36	Programmation	B - comprend cas normaux et limites
37	Algorithmique	A - examine des éléments un à un
38	Algorithmique	B - le premier élément
39	Algorithmique	B - le plus petit élément restant à sa position
40	Algorithmique	D - quadratique
41	Algorithmique	A - une liste triée
42	Algorithmique	B - fait un choix local à chaque étape

Exploitation

- 35 à 42 : socle solide ;
- 28 à 34 : consolidation ciblée ;
- 21 à 27 : reprise structurée ;
- moins de 21 : reconstruire les sept thèmes.

Le score global ne doit pas masquer un thème faible : calculer aussi les sept sous-scores, chacun sur 6.